

Wyszukiwanie informacji o kościołach parafialnych z wykorzystaniem dużych modeli
wizyjnych (Vision LLMs)

Spis treści

1	Zastosowane narzędzia i metoda ekstrakcji danych	5
1.1	Charakterystyka zadania	5
1.2	Zastosowane narzędzia: podejmowane próby i wybrana metoda	7
1.2.1	Metody regułowe i heurystyczne	7
1.2.2	Naiwne użycie gotowego interfejsu LLM	7
1.2.3	Próba z modelem dokumentowym LayoutLMv3	8
1.2.4	Programistyczne przetwarzanie strona po stronie z własnymi promptami . . .	8
1.2.5	Przetwarzanie sekwencyjne z propagacją kontekstu	10
1.3	Architektura hybrydowa: OCR, model dokumentowy i model wizyjny	12
1.4	Opis pakietu danych badawczych	16
1.5	Predykcja	17
1.6	Ewaluacja	18

Rozdział 1

Zastosowane narzędzia i metoda ekstrakcji danych

1.1 Charakterystyka zadania

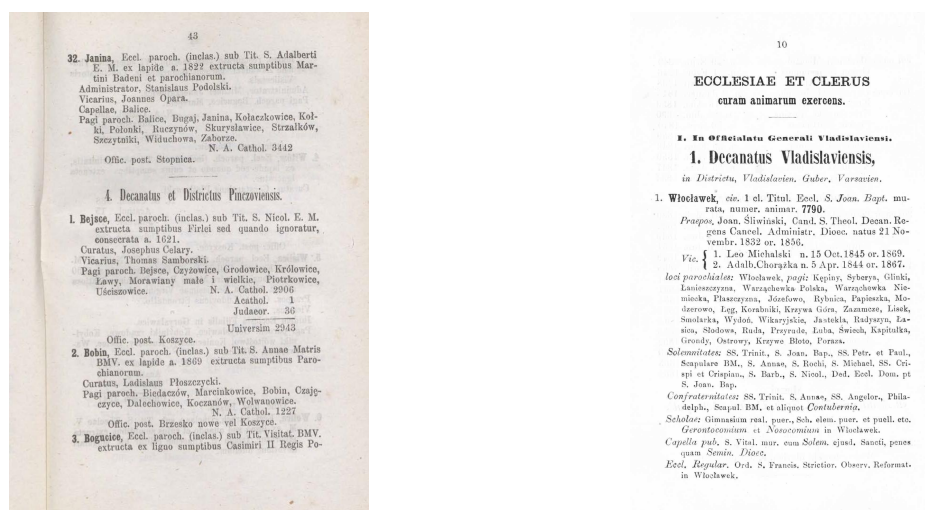
Automatyczna ekstrakcja ustrukturyzowanych danych ze zdigitalizowanych schematyzmów diecezjalnych wpisuje się w klasę problemów określaną w informatyce jako *visual document understanding* (VDU) - zagadnień, w których system komputerowy musi interpretować treść semantyczną obrazów dokumentów, uwzględniając zarówno układ typograficzny, jak i treść językową [9, 5]. W odróżnieniu od generycznego rozpoznawania pisma (OCR), którego celem jest wierne przepisanie ciągłego tekstu, opisywane tu zadanie wymaga od systemu identyfikacji i klasyfikacji konkretnych *pól informacyjnych* osadzonych w hierarchicznej strukturze administracyjnej, a następnie zwrócenia ich w postaci rekordów nadających się bezpośrednio do zasilenia bazy danych.

Zadanie wykazuje jednak własność, która wykracza poza standardowe problemy VDU i którą można określić mianem *zależności od kontekstu międzystroniowego* (*cross-page context dependency*): poprawna wartość pewnych pól na stronie p jest zdeterminowana przez informację widoczną wyłącznie na wcześniejszych stronach $1, \dots, p-1$, nieobecną na aktualnie przetwarzanej stronie. Standardowe benchmarki VDU takie jak DocVQA czy FUNSD przyjmują w swoich założeniach, że cała informacja niezbędna do ekstrakcji danego pola jest widoczna na tej samej stronie [9, 5]. Strukturalnie opisywana własność polega na odnajdywaniu odniesień w poprzedzającym tekście: rekord parafialny na stronie p odsyła do nagłówka dekanatu pełniącego rolę znacznika, który mógł pojawić się wiele stron wcześniej. W odróżnieniu jednak od zagadnień *long document understanding*, w których rozszerza się okno kontekstu modelu, aby objął cały dokument jednocześnie, opisywane tu rozwiązanie opiera się na selektywnej propagacji skondensowanego stanu między kolejno przetwarzanymi stronami, co czyni je bliższym sekwencyjnemu przetwarzaniu ze stanem (*stateful sequential processing*) niż jednoprzebiegowemu odczytowi długiego dokumentu [1, 8].

Na każdej stronie schematyzmu może znajdować się jeden lub kilka opisów parafii, zgrupowanych pod nagłówkami dekanatów, które odpowiadają trójpoziomowej hierarchii Diecezja $\rightarrow$ Dekanat $\rightarrow$ Parafia. Przedmiotem ekstrakcji jest pięć pól dla każdego rekordu parafialnego: (1) numer strony w druku źródłowym; (2) przynależność dekanalna; (3) nazwę parafii; (4) wezwanie kościoła; oraz (5) materiał budowlany obiektu.

Dodatkowe wyzwania wynikają ze specyfiki materiału źródłowego. Dokumenty zostały sporządzone po łacinie z powszechnym użyciem skrótów, których dla pierwszego etapu ekstrakcji, historyczna forma musi zostać zachowana w danych wyjściowych bez jakichkolwiek zmian (np. *S. Martini E.C., B.M.V., mur., ligna.*).

Niezależnie od wierności wobec tekstu źródłowego, system musi radzić sobie także z dużym zróżnicowaniem jakości skanów. Mimo dobrze przeprowadzonemu procesowi digitalizacji, wiele wydań z dwudziestolecia międzywojennego współlistniejących w korpusie danych z zniszczonymi woluminami z połowy XIX wieku, charakteryzuje się nieregularnym składem, plamami i wypłowiałym atramentem. Z kolei sam układ stron waha się od pojedynczej kolumny narracyjnych opisów po gęste, dwukolumnowe zestawienia tabelaryczne, a adnotacje marginalne muszą być niezawodnie odróżniane od treści głównej. Zestawienie dwóch reprezentatywnych przykładów takiego zróżnicowania pokazano na rys. 1.1.



Rysunek 1.1: Dwa przykładowe układy i stany zachowania stron schematyzmów wykorzystanych w badaniu. Po lewej widoczna jest karta o słabszej jakości i bardziej nieregularnym składzie, po prawej strona czytelniejsza, otwierająca nową sekcję dekanalną.

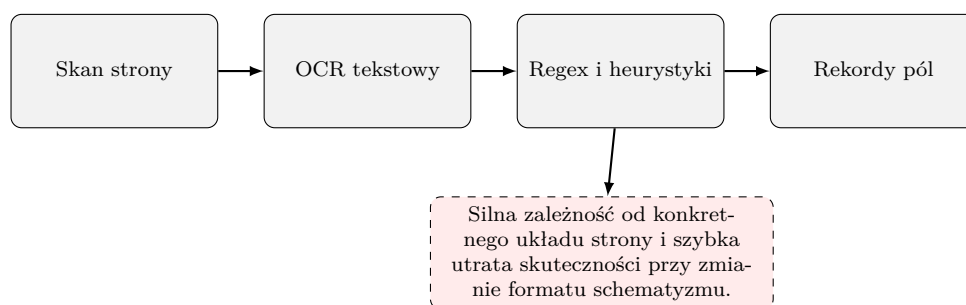
Wszystkie te cechy czynią z opisywanego zadania wymagający przypadek ekstrakcji informacji z dokumentów na poziomie całego dokumentu, stosowanej w specjalistycznej dziedzinie historycznej.

1.2 Zastosowane narzędzia: podejmowane próby i wybrana metoda

Prace nad automatyczną ekstrakcją danych ze schematyzmów nie przebiegały liniowo. Kolejne podejścia wynikały z rozpoznania ograniczeń poprzednich metod: od prób deterministycznych, przez eksperymenty z gotowymi interfejsami modeli, po własny potok przetwarzania oparty na sekwencyjnym wywoływaniu modelu wielomodalnego.

1.2.1 Metody regułowe i heurystyczne

Pierwsze eksperymenty miały charakter klasyczny: punktem wyjścia był tekst OCR, na którym próbowano budować ekstrakcję z użyciem wyrażeń regularnych, podziału na wiersze, prostych heurystyk interpunkcyjnych oraz reguł rozpoznających nagłówki i skróty. Podejście to było atrakcyjne z inżynierskiego punktu widzenia, ponieważ dawało pełną kontrolę nad każdym etapem przetwarzania i nie wymagało kosztownych wywołań modeli generatywnych. W praktyce okazało się jednak zbyt kruche. Schematyzmy różniły się nie tylko językiem i zasobem skrótów, ale również liczbą kolumn, szerokością łamu, sposobem zapisu nagłówków dekanalnych oraz długością pojedynczych wpisów parafialnych. Reguły działające poprawnie dla jednego typu układu bardzo szybko zawodziły po przejściu do innego rocznika lub diecezji, dlatego rozwiązanie to uznano za niewystarczające jako podstawa całego systemu. Schemat ideowy tego etapu pokazuje rys. 1.2.

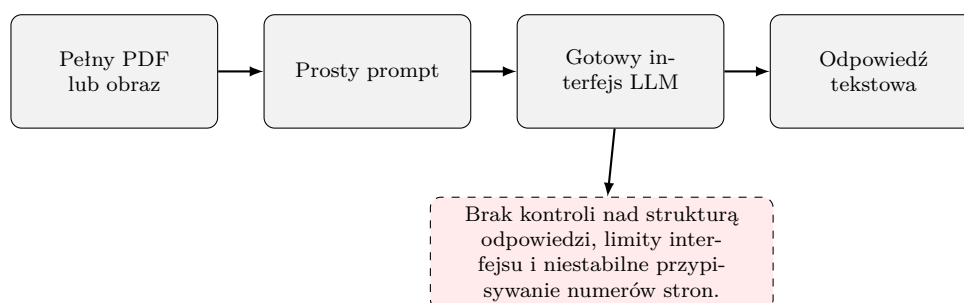


Rysunek 1.2: Podejście regułowe: ekstrakcja oparta na OCR i zestawie ręcznie definiowanych heurystyk.

1.2.2 Naiwne użycie gotowego interfejsu LLM

Kolejnym krokiem było sprawdzenie, czy zadanie da się rozwiązać szybciej przy użyciu gotowego interfejsu czatowego dużego modelu językowego lub wizyjnego. W tym wariancie do modelu przesyłano cały plik PDF schematyzmu albo pojedynczy obraz wraz z prostym poleceniem wyodrębnienia danych parafialnych. Takie rozwiązanie miało wartość rozpoznawczą: pozwalało szybko ocenić, czy współczesne modele w ogóle potrafią zidentyfikować interesujące nas pola. Wyniki okazały się jednak niestabilne jakościowo. Model traktował dokument jako jednolity strumień treści, słabo zachowywał odniesienie do fizycznych stron źródła i często błędnie przypisywał numer strony do wyodrębnionych parafii. Dodatkowym ograniczeniem był brak kontroli nad formatem odpowiedzi oraz limity długości plików i odpowiedzi narzucane przez gotowy interfejs. Ogólny schemat tego etapu przedstawia

rys. 1.3.



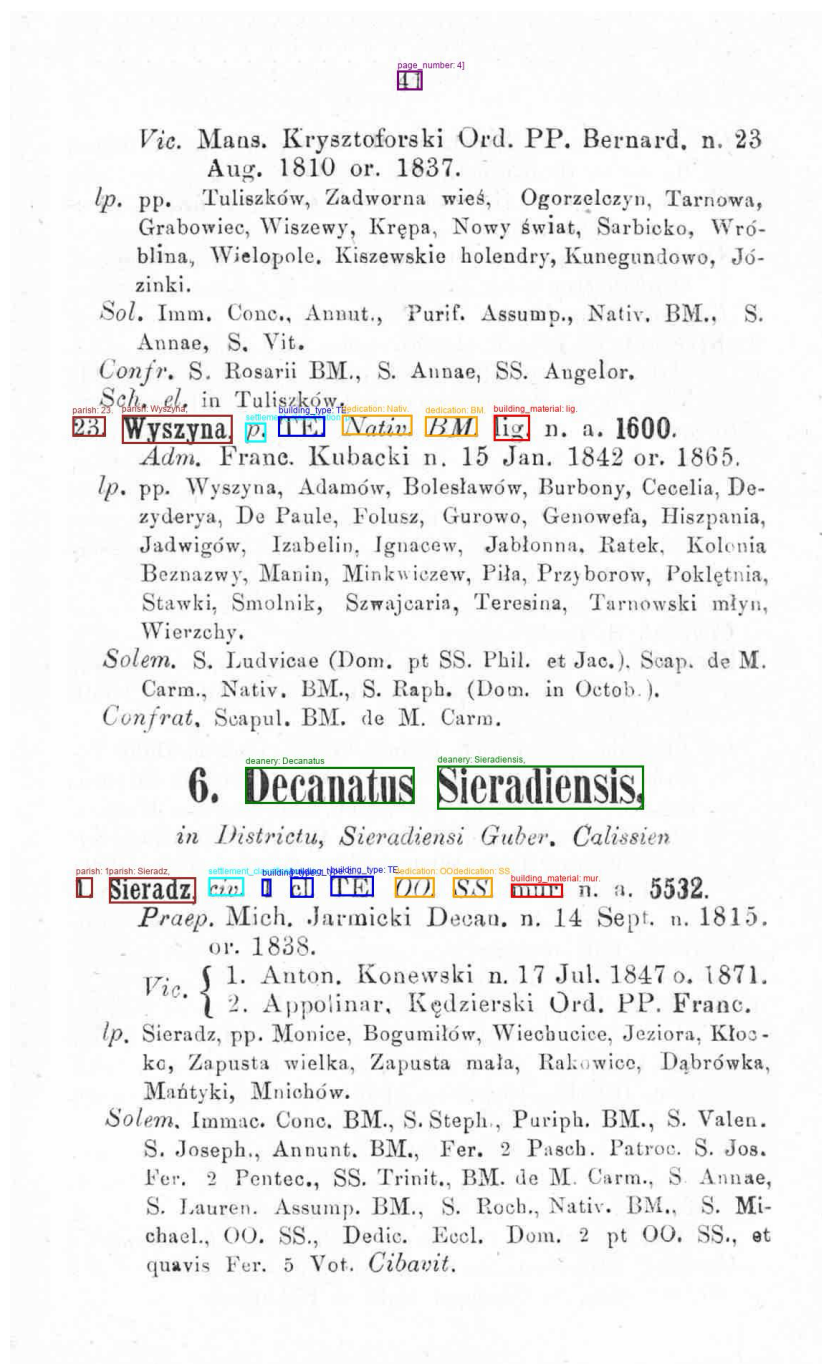
Rysunek 1.3: Naiwne użycie modelu w gotowym interfejsie czatowym.

1.2.3 Próba z modelem dokumentowym LayoutLMv3

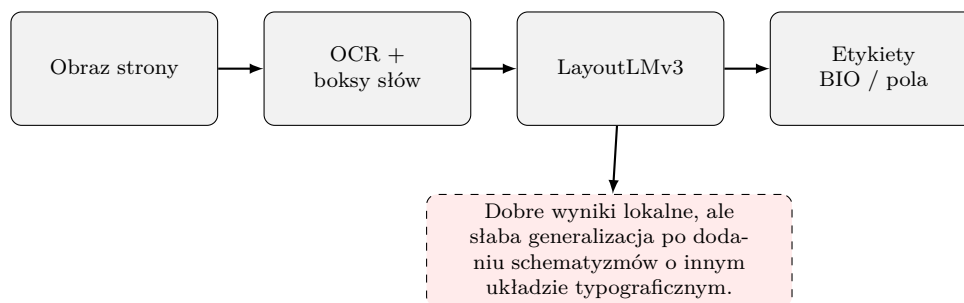
Równolegle podjęto próbę budowy bardziej klasycznego rozwiązania uczenia maszynowego, opartego na modelu LayoutLMv3 firmy Microsoft, wyspecjalizowanym w rozumieniu układu dokumentów [3]. Model ten trenowano na ręcznie anotowanych stronach w notacji BIO (*Beginning–Inside–Outside*), standardowo stosowanej w zadaniach rozpoznawania encji nazewniczych. W tym schemacie każdy token otrzymuje etykietę oznaczającą początek encji danego typu, kontynuację tej encji albo brak przynależności do jakiegokolwiek klasy. W badanym wariancie anotacje obejmowały siedem klas: numer strony, materiał budowlany, klasyfikację miejscowości, parafę, typ obiektu sakralnego, wezwanie oraz dekanat. Klasa klasyfikacji miejscowości została jednak ostatecznie pominięta w końcowym schemacie ekstrakcji, dlatego w późniejszym rozwiązaniu wykorzystywano już tylko numer strony oraz pięć klas właściwego rekordu. W obrębie schematyzmów podobnych do tych, które znalazły się w zbiorze treningowym, wyniki były obiecujące. Problem pojawiał się w momencie rozszerzania zestawu danych o źródła o odmiennej strukturze graficznej. Włączenie do treningu schematyzmów o innym kształcie łamu, innej organizacji kolumn lub odmiennym sposobie zapisu wpisów powodowało wyraźny spadek jakości predykcji. Oznaczało to, że model nie generalizował dostatecznie dobrze na heterogeniczny korpus, który był dla projektu kluczowy. Z tego względu LayoutLMv3 nie został przyjęty jako rozwiązanie finalne; w późniejszej architekturze pełnił jedynie rolę pomocniczego generatora podpowiedzi. Przykład wizualizacji predykcji tego modelu pokazano na rys. 1.4, natomiast ogólny schemat tego etapu przedstawia rys. 1.5.

1.2.4 Programistyczne przetwarzanie strona po stronie z własnymi promptami

Po doświadczeniach z gotowym interfejsem oraz z modelem dokumentowym zdecydowano się na zbudowanie własnego potoku programistycznego. Dokument PDF dzielono na obrazy pojedynczych stron, a następnie dla każdej z nich wykonywano osobne wywołanie modelu wielomodalnego. Kluczowa różnica polegała na tym, że zapytanie nie było już swobodną wiadomością wpisywaną w interfejs, lecz kontrolowaną konstrukcją złożoną z własnego promptu systemowego, własnego promptu użytkownika oraz oczekiwanego formatu odpowiedzi. To podejście pozwoliło wymusić bardziej stabilny zwrot danych w postaci ustrukturyzowanej i znacząco poprawiło związek wyniku z konkretną

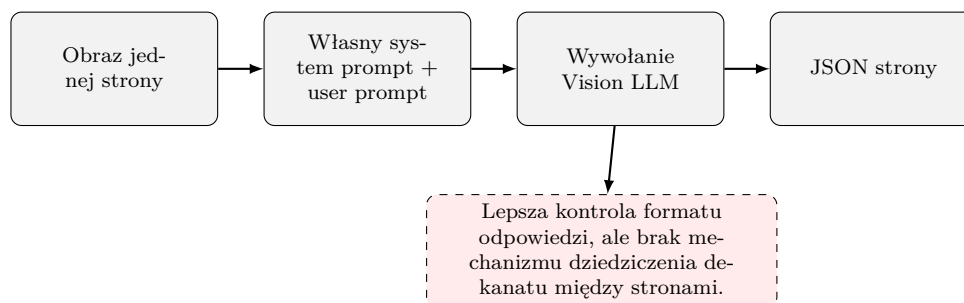


Rysunek 1.4: Wizualizacja przykładowych predykcji modelu LayoutLMv3 na stronie schematyzmu. Kolorowe ramki pokazują tokeny przypisane przez model do klas używanych na etapie treningu, w tym numeru strony, nazwy parafii, dekanatu, wezwania, typu obiektu, materiału budowlanego oraz klasyfikacji miejscowości.



Rysunek 1.5: Próba oparcia ekstrakcji na wytrenowanym modelu LayoutLMv3.

stroną źródła. Jednocześnie odsłoniło ono główny problem merytoryczny całego zadania: jeśli dekanat nie był jawnie widoczny na analizowanej stronie, model wciąż nie miał podstaw, aby poprawnie go odziedziczyć. Schemat tego etapu pokazuje rys. 1.6.

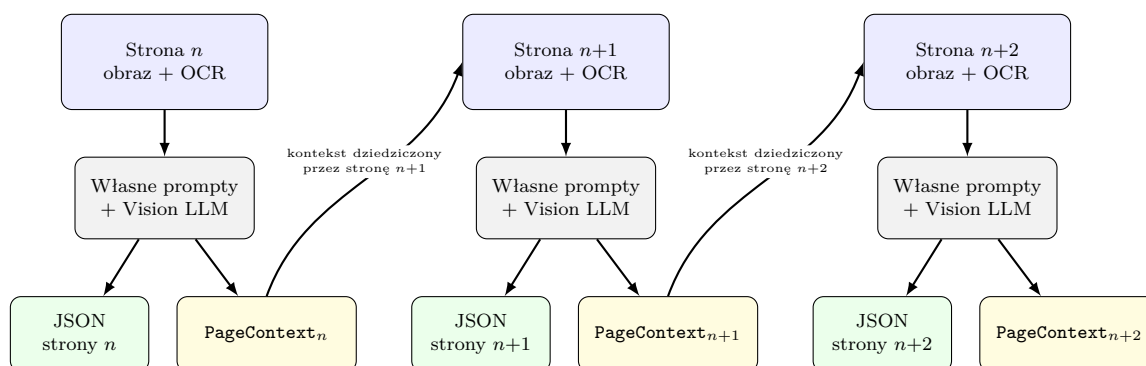


Rysunek 1.6: Programistyczne wywołanie modelu dla pojedynczej strony z użyciem własnych promptów.

1.2.5 Przetwarzanie sekwencyjne z propagacją kontekstu

Rozwiązanie przyjęte na ostatnim etapie eksperymentów zachowało zalety własnego potoku programistycznego, ale zostało uzupełnione o mechanizm propagacji stanu między kolejnymi stronami. Po przetworzeniu każdej strony model zwraca, obok właściwych rekordów parafialnych, obiekt kontekstu zawierający aktualnie obowiązujący dekanat, numer ostatnio przetworzonej strony, streszczenie dotychczasowej zawartości dokumentu oraz opcjonalną adnotację sygnalizującą niską pewność odczytu. Przy przetwarzaniu kolejnej strony obiekt ten jest włączany do treści następnego zapytania, dzięki czemu model otrzymuje informację pomocną przy dziedziczeniu przynależności dekanalnej nawet wówczas, gdy nagłówek dekanatu nie pojawia się na bieżącej stronie. Własne prompty systemowy i użytkownika zostały na tym etapie dopracowane tak, aby precyzyjnie definiowały rolę modelu, zasady dziedziczenia dekanatu, reguły odczytu układów wielokolumnowych oraz oczekiwany format JSON. Ważnym uzupełnieniem stało się także dołączanie tekstu OCR strony następnej, który pomaga domknąć wpisy urwane u dołu bieżącej karty. Schemat wybranego rozwiązania pokazano na rys. 1.7.

W praktyce system zarządza historią konwersacji na dwa sposoby. W wariancie akumulacyjnym przechowywana jest pełna historia wymiany z modelem, z której usuwane są jedynie obrazy i tekst OCR kolejnych stron, aby ograniczyć objętość przy zachowaniu ciągłości merytorycznej. W wariancie okna przesuwającego historia jest przycinana do ustalonej liczby ostatnich wymian, a długoterminowy kontekst przenoszony jest wyłącznie przez obiekt kontekstu. To właśnie ten etap został przyjęty jako architektura rozwiązania końcowego, choć dalsza część artykułu pokazuje, że uzyskana skuteczność pozostała wyraźnie zróżnicowana między polami.



Rysunek 1.7: Rozwiązanie końcowe: sekwencyjne przetwarzanie stron n , $n+1$ i $n+2$, w którym każda odpowiedź generuje obiekt `PageContext` przekazywany do następnego kroku.

1.3 Architektura hybrydowa: OCR, model dokumentowy i model wizyjny

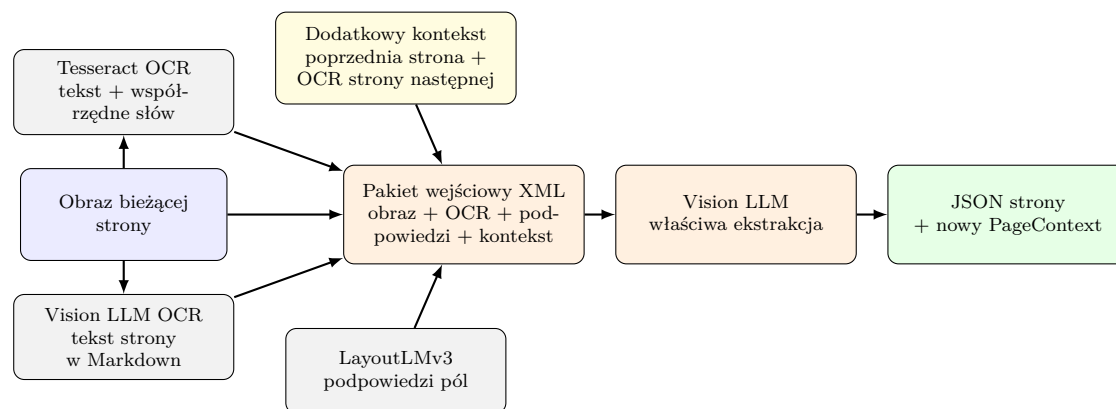
Ostatecznie przyjęta metoda opiera się na trzech komponentach działających sekwencyjnie dla każdej strony.

Pierwszy stanowi optyczne rozpoznawanie znaków (OCR). Początkowo wykorzystywano w tym celu silnik Tesseract, skonfigurowany do obsługi trzech alfabetów (łacińskiego, polskiego i rosyjskiego), który dostarcza surowy tekst strony wraz ze współrzędnymi każdego rozpoznanego słowa na obrazie [11]. Tesseract zwraca jednak jednorodny strumień tekstu, pozbawiony informacji o strukturze typograficznej oryginału: pogrubienia nagłówków dekanatów, tabelaryczne zestawienia danych parafialnych czy podziały na kolumny zostają utracone. Tymczasem cechy te okazały się istotne zarówno dla poprawnej interpretacji źródła, jak i dla skuteczności dalszych etapów ekstrakcji. Potrzeba ich zachowania skłoniła do zastosowania na etapie OCR również modelu wizyjnego (*Vision LLM*), który konwertuje skan strony na tekst w formacie Markdown wiernie odwzorowującym układ druku: nagłówki, pogrubienia, podziały na akapity, tabele i strukturę kolumnową. Rozwiązanie to jest znacząco bardziej wymagające obliczeniowo od klasycznego OCR, lecz dostarcza reprezentację tekstową bogatszą i bliższą oryginałowi. W ostatecznej wersji systemu Tesseract pozostaje w użyciu wyłącznie jako źródło współrzędnych słów niezbędnych dla modelu *LayoutLMv3*, natomiast tekst strony przekazywany do głównego modelu ekstrakcji pochodzi z OCR realizowanego przez model wizyjny. Żaden z wyników OCR nie jest traktowany jako autorytatywny. Oba pełnią rolę pomocniczą wobec obrazu strony, który pozostaje pierwotnym źródłem informacji.

Drugim komponentem jest model *LayoutLMv3* firmy Microsoft, wyspecjalizowany w rozumieniu układu dokumentów [3]. W ramach projektu podjęto próbę wytrenowania tego modelu jako samodzielnego narzędzia ekstrakcji. W tym celu postawiono instancję platformy Label Studio z dedykowanym backendem uczenia maszynowego, umożliwiającym wstępne adnotowanie stron za pomocą bieżącej wersji modelu i ich ręczną korektę przez anotatorów [4]. Zbiór treningowy objął dziesięć schematyzmów o zróżnicowanym charakterze, a model trenowano z użyciem notacji BIO na siedmiu klasach encji: parafia, dekanat, wezwanie, materiał budowlany, numer strony, typ obiektu oraz klasyfikacja miejscowości, stosując funkcję straty Focal Loss w celu zrównoważenia silnie nierównomiernego rozkładu klas [7]. Ostatnia z wymienionych klas nie została jednak wykorzystana w końcowym schemacie ekstrakcji przyjętym dla rozwiązania docelowego. Wyniki na schematyzmach o układzie zbliżonym do danych treningowych były obiecujące, jednak dołączenie schematyzmów o odmiennej strukturze typograficznej powodowało wyraźny spadek jakości, ponieważ model nie generalizował dostatecznie dobrze na różnorodność formatów obecnych w korpusie. Ponadto architektura *LayoutLMv3* przetwarza każdą stronę niezależnie i nie oferuje mechanizmu propagacji kontekstu między stronami, co uniemożliwiało rozwiązanie problemu dziedziczenia przynależności dekanalnej opisanego w sekcji 1.1. Z tych powodów model ten nie został przyjęty jako samodzielne narzędzie ekstrakcji, lecz pełni rolę opcjonalnego generatora odpowiedzi: na podstawie obrazu strony i danych z Tesseracta oznacza poszczególne słowa etykietami pól, a wynik ten przekazywany jest do głównego modelu wizyjnego jako zestaw niepewnych sugestii wymagających weryfikacji na podstawie obrazu.

Trzeci i centralny komponent stanowi wielomodalny duży model językowy (*Vision LLM*), zdolny do jednoczesnej analizy obrazu strony i towarzyszącego mu tekstu. Model ten otrzymuje: obraz bieżącej strony jako pierwotne źródło informacji, tekst OCR bieżącej strony jako wsparcie, opcjonalne odpowiedzi z modelu *LayoutLMv3*, kontekst z poprzedniej strony oraz tekst OCR strony następnej, który służy uzupełnieniu wpisów rozpoczynających się na bieżącej stronie, lecz urwanych u

jej dołu. Na podstawie tych danych wejściowych model zwraca ustrukturyzowany obiekt JSON zawierający listę rekordów parafialnych i obiekt kontekstu do przekazania dalej. Na poziomie operacyjnym oznacza to architekturę, w której surowy obraz strony trafia równolegle do dwóch torów OCR: klasycznego Tesseracta oraz OCR realizowanego przez model wizyjny. Niezależnie od tego ten sam obraz wchodzi bezpośrednio do właściwego pakietu wejściowego modelu ekstrakcji, gdzie jest łączony z wynikami OCR, podpowiedziami LayoutLMv3 oraz dodatkowym kontekstem wielostronicowym. Schemat tej architektury przedstawiono na rys. 1.8.



Rysunek 1.8: Pipeline ekstrakcji danych z pojedynczej strony dokumentu.

Zachowanie modelu regulowane jest szczegółową instrukcją systemową o objętości ok. 350 wierszy, obejmującą: definicję roli, reguły ekstrakcji dla każdego pola, zasady odczytu układów jedno- i wielokolumnowych, reguły dziedziczenia kontekstu dekanalnego, dozwolone i zabronione korekty odczytu OCR oraz listę kontrolną jakości wynikowego obiektu JSON. Istotne jest przy tym, że prompty nie miały postaci swobodnego opisu w języku naturalnym, lecz były organizowane w formacie XML-podobnym, z osobnymi blokami dla roli modelu, wejściowego kontekstu wielostronicowego, reguł ekstrakcji oraz schematu odpowiedzi. Analogiczną strukturę miał prompt użytkownika, w którym osobnymi znacznikami przekazywano kontekst poprzedniej strony, podpowiedzi z LayoutLMv3 oraz tekst OCR strony bieżącej i następnej. Instrukcję uzupełniają cztery przykłady wzorcowe (*few-shot examples*), ilustrujące typowe scenariusze: stronę z nagłówkiem dekanatu, nagłówek pojawiający się w środku strony, wiele parafii na jednej stronie oraz stronę niezawierającą rekordów parafialnych.

Rysunki 1.11 i 1.12 pokazują skrócony przykład rzeczywistego pakietu wejściowego oraz odpowiadającej mu odpowiedzi modelu dla strony otwierającej wykaz parafii w schematyzmie wrocławskim z 1872 r.

W toku eksperymentów przetestowano kilkanaście modeli wizyjnych różnych producentów. Ostatecznie do systematycznej ewaluacji na pełnym korpusie wykorzystano model Gemini Flash firmy Google, który okazał się najbardziej obiecującym kompromisem jakości ekstrakcji, kosztu i szybkości przetwarzania [2]. Generowanie odpowiedzi odbywa się przy niskiej wartości parametru losowości (*temperature* = 0,1), co ogranicza niedeterminizm wyników. Jak pokazano w dalszych częściach artykułu, skuteczność tego rozwiązania pozostała jednak silnie zależna od rodzaju pola: najwyższa dla numeru strony i materiału budowlanego, a wyraźnie niższa dla dekanatu.

```

<SYSTEM_PROMPT>
  <MULTI_PAGE_CONTEXT>
    <SUMMARY_FIELD_IMPORTANCE>
      The `summary` field serves as a compressed memory
      of document progression.
    </SUMMARY_FIELD_IMPORTANCE>
  </MULTI_PAGE_CONTEXT>
  <VALUE_EXTRACTION_RULES>
    <DEANERY>
      If context provides active_deanery but no new heading
      appears: use inherited value
    </DEANERY>
  </VALUE_EXTRACTION_RULES>
  <OUTPUT_SCHEMA>
    ...
  </OUTPUT_SCHEMA>
</SYSTEM_PROMPT>

```

Rysunek 1.9: Fragment rzeczywistego promptu systemowego w formacie XML-podobnym. Osobne znaczniki porządkują kontekst wielostronicowy, reguły ekstrakcji oraz oczekiwany schemat odpowiedzi JSON.

```

<USER_REQUEST>
  <PREVIOUS_PAGE_CONTEXT>
    <ACTIVE_DEANERY>Decanatus Wyszynensis</ACTIVE_DEANERY>
    <PREVIOUS_SUMMARY>Page 14: 3 parishes...</PREVIOUS_SUMMARY>
  </PREVIOUS_PAGE_CONTEXT>
  <MODEL_HINTS source="layoutlmv3">
    Suggestions from a smaller model...
  </MODEL_HINTS>
  <CURRENT_PAGE_TEXT>
    OCR text from the current page...
  </CURRENT_PAGE_TEXT>
  <NEXT_PAGE_TEXT>
    OCR text from the next page...
  </NEXT_PAGE_TEXT>
</USER_REQUEST>

```

Rysunek 1.10: Fragment promptu użytkownika. Osobne znaczniki przekazują kontekst odziedziczony z poprzedniej strony, podpowiedzi modelu LayoutLMv3 oraz pomocniczy tekst OCR dla strony bieżącej i następnej.

```
{
  "image_path": ".../e6857f7b020207a54378a0f014c1c470.jpeg",
  "previous_page_context": {
    "active_deanery": null,
    "summary": "Previous pages covered administrative lists
      and seminary details.",
    "last_page_number": "9"
  },
  "current_page_text": "10 ... Decanatus Vladislaviensis ...
    1. 11Wocawek ... Titul. Eccl. S. Joan. Bapt. murata ...",
  "next_page_text": "... continuation of clergy and localities ...",
  "model_hints": "parish: 11Wocawek; dedication: S. Joan. Bapt.;
    building_material: murata"
}
```

Rysunek 1.11: Skrócony przykład pakietu wejściowego przekazywanego do modelu dla pojedynczej strony. Obejmuje on obraz strony, kontekst odziedziczony z poprzedniej karty, tekst OCR oraz opcjonalne podpowiedzi modelu dokumentowego.

```
{
  "page_number": "10",
  "entries": [
    {
      "deanery": "Decanatus Vladislaviensis",
      "parish": "11Wocawek",
      "dedication": "S. Joan. Bapt.",
      "building_material": "murata"
    }
  ],
  "context": {
    "summary": "Page 10: First page of parish records.
      Introduces Decanatus Vladislaviensis.",
    "note": null,
    "active_deanery": "Decanatus Vladislaviensis",
    "last_page_number": "10"
  }
}
```

Rysunek 1.12: Skrócony przykład odpowiedzi modelu dla tej samej strony. Oprócz właściwych rekordów zwracany jest także obiekt kontekstu wykorzystywany przy przetwarzaniu kolejnych stron.

1.4 Opis pakietu danych badawczych

Korpus źródłowy obejmuje 72 schematyzmy diecezjalne z 23 diecezji polskich, reprezentujące dwa główne przedziały chronologiczne: lata sześćdziesiąte-osiemdziesiąte XIX wieku oraz lata trzydzieste XX wieku. Każdy schematyzm został zdigitalizowany w postaci pliku PDF, a następnie podzielony na osobne obrazy stron w wysokiej rozdzielczości, stanowiące podstawowe dane wejściowe systemu ekstrakcji.

Dane referencyjne, względem których oceniana jest jakość automatycznej ekstrakcji, stanowi baza danych zbudowana ręcznie przez zespół historyków w ramach projektu <tutaj dać nazwę projektowi>. Dla każdego schematyzmu baza zawiera sekwencję rekordów parafialnych z wypełnionymi polami: dekanat, parafia, wezwanie i materiał budowlany, przyporządkowanymi do konkretnych stron źródłowych. Zbiór ten, liczący łącznie ok. 35, 000 rekordów, pełni podwójną rolę: jest zarówno docelowym produktem prac dokumentacyjnych projektu głównego, jak i wzorcem (*ground truth*) do ewaluacji narzędzia cyfrowego.

1.5 Predykcja

Wynik automatycznej ekstrakcji dla każdego schematyzmu przyjmuje postać sekwencji rekordów o identycznej strukturze pól co dane referencyjne. Ponieważ model ekstrakcji zwraca wartości w oryginalnej formie źródłowej (łacińskiej, rosyjskiej bądź niemieckiej), a baza referencyjna projektu głównego posługuje się polskimi odpowiednikami i skrótami, konieczny jest dodatkowy etap tłumaczenia, w dalszym ciągu określany jako *parsowanie*. Etap ten opiera się na trzech słownikach mapujących formy źródłowe na znormalizowane wartości polskie: dla materiału budowlanego (np. *mur.* → *mr*, *lign.* → *dr*), dla wezwań kościelnych (np. *Assumptionis B.M.V.* → *Wniebowzięcie NMP*) oraz dla nazw dekanatów (np. *Decanatus Gedanensis* → *Gdańsk*). Aby uwzględnić warianty pisowni i drobne błędy w odczycie, dopasowanie w słownikach odbywa się z wykorzystaniem przybliżonego porównania łańcuchów z konfigurowalnym progiem tolerancji. W rezultacie ewaluacja prowadzona jest dwutorowo: raz na poziomie formy źródłowej (*source*), a raz na poziomie formy przetłumaczonej (*parsed*), co pozwala oddzielnie mierzyć jakość samej ekstrakcji od jakości tłumaczenia na język docelowy bazy. Porównanie obu sekwencji, referencyjnej i wygenerowanej, nie jest jednak trywialne: mogą się one różnić długością (model mógł pominąć pewne parafie lub wygenerować wpisy nieistniejące w źródle), a kolejność rekordów nie musi się pokrywać. Problem ten rozwiązano za pomocą algorytmu węgierskiego, który znajduje optymalne przyporządkowanie parami między rekordami referencyjnymi a przewidywanymi, minimalizując łączną niepodobność. Podobieństwo dwóch rekordów obliczane jest jako średnia ważona podobieństw poszczególnych pól, gdzie każde pole (dekanat, parafia, wezwanie, materiał budowlany) ma przypisaną wagę odzwierciedlającą jego znaczenie. Podobieństwo pojedynczego pola mierzone jest znormalizowaną odległością Levenshteina po uprzednim sprowadzeniu obu wartości do formy porównywalnej: usunięciu znaków diakrytycznych, ujednoliceniu wielkości liter i normalizacji białych znaków. Rekordy, których wzajemne podobieństwo nie przekracza ustalonego progu, nie są łączone w pary i trafiają odpowiednio do kategorii wpisów pominiętych (po stronie referencji) lub nadmiarowych (po stronie predykcji).

1.6 Ewaluacja

Porównanie sekwencji referencyjnej i wygenerowanej rozpoczyna się od dopasowania rekordów za pomocą algorytmu węgierskiego [6]. Macierz podobieństwa budowana jest na podstawie ważonej zgodności pól rekordu, przy czym na etapie samego dopasowania implementacja korzysta ze znormalizowanych łańcuchów znaków i miary podobieństwa `SequenceMatcher` z biblioteki `difflib` [10]. Takie rozwiązanie pozwala zestawiać ze sobą sekwencje różniące się długością i kolejnością wpisów, a jednocześnie zachować próg jakości wymagany do uznania dwóch rekordów za odpowiadające sobie.

Po wykonaniu dopasowania obliczane są dwa typy ocen. W wariancie *source* dla każdego pola raportowane są miary podobieństwa tekstowego: średnia, minimum i maksimum. W tym etapie wykorzystywany jest scorer oparty na znormalizowanym podobieństwie edycyjnym typu Levenshteina [12]. W wariancie *parsed* ten sam typ porównania służy do wyznaczenia poprawności klasyfikacyjnej po przyłożeniu ustalonego progu zgodności. Nie jest to jednak klasyczna macierz pomyłek budowana dla niezależnych obserwacji i zamkniętego zbioru etykiet. Kategorie *true positive*, *false positive*, *false negative* i *true negative* wyznaczane są tu po wcześniejszym dopasowaniu rekordów, a następnie przez sprawdzenie, czy dana wartość pola została odtworzona w stopniu wystarczająco zgodnym z referencją.

Za trafienie (*true positive*) uznaje się sytuację, w której zarówno wartość referencyjna, jak i przewidywana istnieją, a ich podobieństwo przekracza ustalony próg. Wpis wygenerowany przez model, któremu nie odpowiada żadna wartość referencyjna, lub którego wartość odbiega od referencji poniżej progu, liczony jest jako błąd nadmiarowy (*false positive*). Analogicznie, wpis referencyjny bez odpowiadającej mu predykcji lub z predykcją poniżej progu stanowi pominięcie (*false negative*). W szczególności przypadek, w którym model zwrócił wartość dla pola, ale nie spełnia ona progu zgodności względem referencji, traktowany jest jednocześnie jako błąd nadmiarowy i pominięcie, ponieważ system wygenerował odpowiedź, lecz nie odtworzył poprawnej wartości. Na tej podstawie wyznaczone są precyzja, czułość, dokładność oraz ich średnia harmoniczna (miara F1), oddzielnie dla każdego pola i dla każdego schematyzmu.

Wyniki ewaluacji eksportowane są w postaci arkuszy kalkulacyjnych, zawierających dla każdego schematyzmu zestawienie metryk per pole wraz z wierszem podsumowującym. Nazwy plików wynikowych zawierają identyfikator użytego modelu oraz znacznik czasowy, co pozwala na systematyczne porównywanie kolejnych przebiegów eksperymentalnych. Interpretacja tych metryk wymaga jednak rozbicia na poszczególne pola, ponieważ dalsza analiza jakościowa pokazuje wyraźne różnice między relatywnie łatwiejszym numerem strony i materiałem budowlanym a znacznie trudniejszym zadaniem poprawnego odziedziczenia dekanatu.

Bibliografia

- [1] Iz Beltagy, Matthew E. Peters i Arman Cohan. *Longformer: The Long-Document Transformer*. 2020. arXiv: 2004.05150 [cs.CL]. URL: <https://arxiv.org/abs/2004.05150>.
- [2] Google. *Gemini API Models*. 2026. URL: <https://ai.google.dev/gemini-api/docs/models> (term. wiz. 08.03.2026).
- [3] Yupan Huang i in. *LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking*. 2022. arXiv: 2204.08387 [cs.CL]. URL: <https://arxiv.org/abs/2204.08387>.
- [4] HumanSignal. *Label Studio Documentation: Write Your Own ML Backend*. 2026. URL: https://labelstud.io/guide/ml_create (term. wiz. 08.03.2026).
- [5] Guillaume Jaume, Hazim Kemal Ekenel i Jean-Philippe Thiran. „FUNSD: A Dataset for Form Understanding in Noisy Scanned Documents”. W: *Proceedings of the ICDAR 2019 Workshop on Robust Reading Challenge on Scanned Receipts OCR and Information Extraction (SROIE)*. 2019.
- [6] Harold W. Kuhn. „The Hungarian Method for the Assignment Problem”. W: *Naval Research Logistics Quarterly* 2.1–2 (1955), strony 83–97. DOI: 10.1002/nav.3800020109. URL: <https://www.math.toronto.edu/mccann/1855/KuhnNRL55.pdf>.
- [7] Tsung-Yi Lin i in. „Focal Loss for Dense Object Detection”. W: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, strony 2980–2988. URL: https://openaccess.thecvf.com/content_iccv_2017/html/Lin_Focal_Loss_for_ICCV_2017_paper.html.
- [8] Nelson F. Liu i in. *Lost in the Middle: How Language Models Use Long Contexts*. 2023. arXiv: 2307.03172 [cs.CL]. URL: <https://arxiv.org/abs/2307.03172>.
- [9] Minesh Mathew, Dimosthenis Karatzas i C. V. Jawahar. „DocVQA: A Dataset for VQA on Document Images”. W: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. 2021, strony 2200–2209.
- [10] Python Software Foundation. *difflib — Helpers for Computing Deltas*. 2026. URL: <https://docs.python.org/3/library/difflib.html> (term. wiz. 08.03.2026).
- [11] Ray Smith. „An Overview of the Tesseract OCR Engine”. W: *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. 2007, strony 629–633. DOI: 10.1109/ICDAR.2007.4376991. URL: <https://research.google/pubs/an-overview-of-the-tesseract-ocr-engine/>.
- [12] Robert A. Wagner i Michael J. Fischer. „The String-to-String Correction Problem”. W: *Journal of the ACM* 21.1 (1974), strony 168–173. DOI: 10.1145/321796.321811.